Your University

Your Department / School

Reinforcement Learning with NEAT in Multitask Environments

Master Thesis

Your Program

Sven Goerdes

Supervisor: Prof. Your Supervisor

Co-supervisor: Dr. Your Co-supervisor

July 15, 2026

Abstract

Standard NEAT holds one activation function fixed across every node. Heterogeneous Activation NEAT (HA-NEAT) makes that function an evolvable per-neuron property. Its reported advantages, however, come only from static classification and regression benchmarks. This thesis asks whether they transfer to multi-task continuous control. There, a single shared genome must drive two Brax locomotion bodies whose observation and action spaces differ in size: Hopper (11 observations, 3 actions) and Walker2D (17 observations, 6 actions). We build one network over the union of both interfaces, with 17 input and 6 output nodes. We then fit each environment to that network by zero-padding Hopper's observation and slicing its actions from the first three outputs, so that no part of the architecture is reserved for either task. Selection uses the normalised minimum of the two returns, which ties a genome's fitness to its weaker task. We compare three conditions: standard NEAT, HA-NEAT, and a tanh-only HA-NEAT ablation that keeps the full mutation mechanism but has no activation diversity. Each ran at population 200 over 750 generations across 30 seeds, for 90 runs in total on the JAX, Brax, and TensorNEAT stack, and we re-evaluated every champion over 20 held-out episodes per task.

Our main result is a null result. Median fitness was 0.184 (NEAT), 0.208 (HA-NEAT), and 0.199 (ablation), with no difference across conditions (Kruskal–Wallis $H = 1.48$, $p = 0.476$). The design rules out only effects of Cohen's $d \geq 0.74$, so smaller differences could have gone undetected. HA-NEAT champions drew on the full activation set, and tanh was their *least* common hidden activation at 11%. We found one structural signal: the ablation's champions are more compact than NEAT's ($p = 0.030$). That effect appears in the condition with no activation diversity at all, which points to the historical markers that HA-NEAT reassigns after an activation change as the likely cause, and away from heterogeneity. One caveat limits these conclusions. Walker2D binds the minimum for 83–97% of champions while Hopper saturates near 0.34 normalised, so the objective degenerated toward single-task Walker2D optimisation and applied less multi-task pressure than the design intended.

Contents

1 Introduction **4**

2 Literature Review **9**

2.1 Structural Differences in the Brain and in Artificial Neural Networks 9

2.2 Neuroevolution and the TWEANN Problem 10

2.3 Neuroevolution of Augmenting Topologies (NEAT) 11

2.4 Activation Functions 12

2.5 Heterogeneous Neuroevolution (HA-NEAT) 14

2.6 Direct and Indirect Encoding 14

2.7 Continuous-Control Benchmarks 15

2.8 Multi-Task Learning with a Single Shared Network 16

2.9 Research Gap 17

3 Methodology **17**

3.1 Problem Formulation 17

3.2 Genome Representation and the Padding Scheme 18

3.3 Algorithm Design: HA-NEAT 20

3.3.1 NEAT Foundation 20

3.3.2 HA-NEAT Extensions 21

3.4 Fitness Evaluation 22

3.5 Network Architecture 24

3.6 Implementation Stack 25

4 Experimental Settings **26**

4.1 Environment Details 26

4.2 Experimental Conditions 27

4.3 Hyperparameters 28

4.4 Preliminary Calibration Experiments 29

4.5 Evaluation Protocol 29

4.6 Compute Resources 32

5 Results and Discussion **33**

5.1 Experimental overview 33

5.2 Multi-task fitness across conditions 34

5.3 Genome complexity and generational dynamics 38

5.4 Per-task trade-off and multi-task pressure 40

5.5 Activation-function usage within HA-NEAT 41

5.6 Discussion 42

6 Conclusion and Future Work **44**

1 Introduction

Biological nervous systems do something artificial neural networks still struggle with. A single brain controls walking, reaching, and balance through shared circuitry, and switches between those skills without being retrained from scratch. It also keeps rewiring itself: mammalian brains form new connections and pathways throughout life, adapting continually to their environment, and this neuroplasticity underlies much of an animal's ability to generalise to situations it has not seen before (Pascual-Leone et al., 2005). The standard practice in deep learning is almost the opposite. We pick a fixed network architecture, apply it to one environment, train until convergence, and start again for the next task. These networks also stack identical neurons that all share one nonlinearity. We ask whether that uniform choice limits how well a single network can hold several skills at once, and whether different activation functions in different neurons would help.

Neuroevolution makes the architecture itself part of what evolves, so the search discovers a network's structure. The method evolves a population of networks, adding nodes and connections and letting the structural variants compete under selection. A genome describes each network: a list of genes recording which nodes exist and how they connect. Mutation edits that list, and selection keeps the genomes whose networks perform best. Topology and weights therefore evolve together. NEAT (Neuroevolution of Augmenting Topologies) is the foundational example (Stanley & Miikkulainen, 2002). It starts from minimal networks and adds nodes and connections through random structural mutation. A speciation mechanism shields each new structure from competition long enough for its weights to adapt, and selection then eliminates the structure that has not proved useful.

One thing standard NEAT still holds constant is the activation function. Every neuron in classical NEAT uses the same activation function, typically tanh. This choice assumes that any function the task needs can be built by combining tanh units. HA-NEAT (Heterogeneous Activation NEAT) drops that assumption (Hagg et al., 2017). Each node's activation function becomes part of its gene and can therefore mutate, so a population can discover that one neuron should behave like tanh, another like ReLU, and a third like a sine. What happens around that activation gene is as critical as the gene itself. NEAT records where every connection first appeared and uses this record to measure how similar two networks are. Networks that share few connections are placed in separate species. When a node's activation function mutates, HA-NEAT marks the connections attached to that node as new, so the mutant appears structurally different from its relatives. The mutant is therefore more likely

to be placed in a new species, where it is shielded from direct competition while its weights adapt. This shielding is the mechanism's intended behaviour (Hagg et al., 2017). How often the protection triggers, and how long it holds, depends on the compatibility threshold and on genome size. Activation diversity and speciation are therefore linked. The mechanism that introduces a new activation function also protects it.

Whether this protection matters depends on the setting. When one shared genome controls several tasks, more function shapes become useful, and a new activation mutation needs time to show its benefit. The speciation coupling gives it that time. We test this on two locomotion benchmarks: Hopper and Walker2D, from the Brax physics suite (Freeman et al., 2021). Hopper is a one-legged body that must hop forward without falling. Walker2D is a two-legged body that must walk. For a genome that must control both, the assumption that one nonlinearity covers every useful function shape is an empirical claim.

However, the available literature does not answer it. NEAT has been studied extensively in single-task settings, and there is related work on co-evolution, modular networks, and transfer between related tasks. The case we care about is missing. One genome evolved for solving two environments whose input and output spaces have different sizes, with no per-task heads and no fine-tuning. Within neuroevolution, even the basic step of passing observations of different sizes through one network is rarely treated directly in the literature we surveyed. Whether activation heterogeneity helps such a shared genome generalise has, to our knowledge, not been tested empirically.

Evolution acts on one selection pressure, so the two per-task returns must be collapsed into a single scalar. That collapse is fitness aggregation. Multi-objective methods such as NSGA-II keep the objectives separate and return a Pareto front of trade-offs (Deb et al., 2002). We stay single-objective for two reasons. Our question is whether one genome can be competent on both tasks at once, not how the two returns trade off against each other. A single scalar is also what NEAT's selection and speciation mechanism consumes unchanged. We compared three scalar aggregators: a normalised minimum (maximin), a normalised sum, and a normalised product.

This thesis takes that as the open question and addresses it. Two research questions guide the work.

- Can a single NEAT genome, shared across Hopper (11 observations, 3 actions) and Walker2D (17 observations, 6 actions) through zero-padded observations and sliced

actions, learn control policies for both tasks simultaneously?

- Does HA-NEAT improve multi-task performance over standard NEAT? And if it does, is the improvement actually about activation diversity, or about the speciation mechanism that comes with it?

A third design choice sits underneath both questions: how to combine two per-task rewards into one selection signal. We treat this as a design decision to be justified, and answer it before the main experiments begin. A preliminary calibration study (Section 4.4) compares the normalised sum, product, and minimum, and the main experiments use the aggregation it selects.

The contribution is empirical. We propose no new optimiser. We take two existing algorithms, one extension, and one experimental setting, and report what happens when they meet.

A shared-genome input/output mapping scheme. The first contribution is the mechanism that lets one NEAT genome run on two environments of different shape, with no per-task heads and no task conditioning. Hopper and Walker2D do not share an interface: Hopper exposes an 11-dimensional observation and a 3-dimensional action, Walker2D a 17-dimensional observation and a 6-dimensional action space. We build a single network over the union of both spaces, with 17 input nodes and 6 output nodes sized to the larger task, and fit each environment to that one interface. A Hopper observation is zero-padded from 11 to 17 dimensions before entering the network, and only the first 3 of the 6 outputs drive its actuators. A Walker2D observation uses all 17 inputs and all 6 outputs directly (Section 3.2). No part of the architecture is reserved for one task, so any shared structure the network develops must come from selection pressure. The network is deliberately small: it is the simplest setup in which we can ask whether one genome generalises across two bodies, without explicit pre-design. The design rests on the assumption that the two bodies are similar enough in shape for shared control strategies to be reachable: each of Walker2D's legs repeats Hopper's segment-and-joint layout (Section 3.1).

A comparison of NEAT and HA-NEAT on a multi-task problem. The second contribution is the controlled experiment this interface enables. We evolve standard NEAT, with every hidden node fixed to tanh, and HA-NEAT, with each hidden node drawing its

activation from a set of functions. Both run on the same Hopper-plus-Walker2D task, under matched population sizes, generation budgets, and seed counts, so the one thing that changes between them is whether evolution can vary the activation function. We fixed the set of functions in advance and did not tune its composition. The set contains several distinct function shapes: saturating (tanh and the sigmoid σ), piecewise-linear (ReLU), periodic (sin), and linear (identity). Selection uses the normalised minimum of the two per-task returns, $\min(r_{\text{Hopper}}/3000, r_{\text{Walker2D}}/5000)$. This ties a genome's fitness to its weaker task, so a specialist that solves one body and fails on the other earns a low score (Section 3.4). The comparison runs on a GPU-native JAX/Brax stack (Freeman et al., 2021; Wang et al., 2024) that compiles the whole evolutionary loop, from mutation and crossover through speciation into a single computation graph. This makes population-based search affordable at the scale the analysis requires: a population of 200 evolved for 750 generations across 30 independent seeds per condition. Every champion, the best genome of a run, is then re-evaluated over 20 held-out episodes per task, so no reported number depends on a single episode. With 30 seeds per condition the design detects differences between the three conditions of Cohen's $d \geq 0.74$, roughly 0.06 normalised-fitness units (Section 5.2). An absence of difference therefore rules out large effects. Smaller ones could go undetected.

An ablation separating activation diversity from speciation protection. HA-NEAT (Hagg et al., 2017) differs from standard NEAT (Stanley & Miikkulainen, 2002) in two coupled ways. It provides a mathematically diverse activation set and applies a structural mutation operator intended to protect activation changes. The operator reassigns the historical markers of every connection attached to the mutated node. These new markings invoke NEAT's speciation criterion, separating the mutant while its weights co-adapt. Because HA-NEAT introduces both mechanisms together, comparisons with standard NEAT cannot determine which mechanism accounts for any performance or parsimony advantage. We introduce a third experimental condition to separate these effects. It retains the HA-NEAT structural mutation and connection-reassignment operator while limiting the activation set to tanh. This lets activation mutation change connection markers even when every node uses tanh. The resulting condition separates structural protection from activation diversity. Because the original HA-NEAT study evaluated only the full algorithm (Hagg et al., 2017), our three-condition design tests the contribution of each mechanism to performance.

The remainder of the thesis is organised as follows. Section 2 reviews neuroevolution, activation heterogeneity, and multi-task control, and locates the gap this work addresses. Section 3

specifies the shared-network encoding, the HA-NEAT extensions, and the fitness aggregation. Section 4 gives the environments, conditions, hyperparameters, calibration, and evaluation protocol. Section 5 reports the multi-task comparison and its follow-up analyses, and discusses what they imply for the questions above.

2 Literature Review

2.1 Structural Differences in the Brain and in Artificial Neural Networks

Unlike Artificial Neural Networks (ANNs), the human brain is a non-static and continuously changing network. Neural cells form new connections throughout life and rewire to find new pathways that better represent the environment the body is experiencing (Holtmaat & Svoboda, 2009; Pascual-Leone et al., 2005). Human learning yields measurable structural plasticity, demonstrating that adaptation is supported by tangible architectural reconfiguration of the brain (Draganski et al., 2004).

In contrast, ANNs typically rely on static connections. We create the structure of the neural network before we start training it. Hence, the topology of the ANN is fixed. Therefore, it is not possible for the ANN to create new connections while being trained. In deep learning we rely on adapting the weights of the ANN through a process called backpropagation (Rumelhart et al., 1986). Inspired by biological learning, backpropagation can be seen as a simplified mathematical way to change synaptic strengths: the network keeps its wiring, but the "strength" of each connection is adjusted during training. However, classical approaches with backpropagation do not allow us to adjust and change the topology of the ANNs.

Biological neurons also differ in their built-in response. Two neurons can receive the same input and react differently, because their internal properties determine how each one transforms it (Drion et al., 2015). This variety is functionally useful. A population of differently responding neurons encodes information more effectively than a uniform one (Tripathy et al., 2013), and adding similar variety to artificial networks improves how well they learn (Perez-Nieves et al., 2021). A single activation function shared across every node discards this source of variety.

2.2 Neuroevolution and the TWEANN Problem

An alternative to gradient-based training is neuroevolution, which replaces backpropagation with an evolutionary search over a population of candidate networks (Stanley et al., 2019). Each generation, the fitter networks reproduce and the weaker ones are discarded. Two operators generate the next generation. Mutation randomly modifies a single parent's genome. In Neuroevolution it changes connection weights or inserts new nodes and connections into the ANN. It is the only operator that extends topology. Crossover recombines two parent genomes into offspring, so that each child inherits structure from both parents. Selection, mutation, and crossover together are the three instruments that drive the population toward better-performing networks. Because the topology is part of the genome, it evolves jointly with the weights. Methods that do so are called Topology and Weight Evolving Artificial Neural Networks (TWEANNs) (Stanley & Miikkulainen, 2002).

A central obstacle for any method that evolves topology is that the same network function can be encoded by genomes whose nodes appear in different orders. Because crossover recombines genomes by position, it can produce offspring that lose structures both parents possessed or that duplicate them incoherently. Recombination then destroys the innovations it should preserve. A method therefore needs a way to identify matching structure before combining genomes (Stanley & Miikkulainen, 2002).

A second obstacle is the survival of new structure. When a mutation adds a node or a connection, it almost always lowers fitness at first, because the newly added weights have not yet been optimised. Such an innovation is out-competed and discarded before it can be tuned, even when the topology it introduces would ultimately prove valuable. A TWEANN method therefore needs a way to give new structure time to mature before selection can eliminate it.

A third problem is that evolved topologies can grow without bound. Many early TWEANNs added penalty terms to discourage bloat. Such penalties, however, are hard to tune and risk suppressing useful growth along with wasteful growth. We therefore grow networks from minimal structure, so that complexity accumulates only where selection retains it.

Earlier TWEANN methods addressed these three problems only in part. Neuroevolution of Augmenting Topologies (NEAT) (Stanley & Miikkulainen, 2002) addresses all three at once, with a single coherent set of mechanisms.

2.3 Neuroevolution of Augmenting Topologies (NEAT)

Neuroevolution of Augmenting Topologies introduces a dynamic evolutionary based framework that optimises both weights and network topology simultaneously (Stanley & Miikkulainen, 2002). It allows topological structure to emerge through mutation and starts out with a population of uniform ANNs with zero hidden nodes. The population therefore begins with minimal networks, in which inputs connect directly to outputs and no hidden nodes exist. Structural mutations then add nodes and connections over generations. The search therefore starts in a low-dimensional weight space and expands only where selection retains the added structure. This approach ensures that the algorithm starts searching for solutions in low dimensions first.

NEAT resolves the three obstacles above with three mechanisms. The first is historical marking. Every gene receives a global innovation number at the moment a structural mutation creates it, and this number is inherited unchanged thereafter. Two genes that share an innovation number encode the same structural feature, even when they sit at different positions in their respective genomes. Crossover therefore aligns parents through their shared innovation numbers. Matching genes are inherited from either parent at random, while disjoint and excess genes are taken from the fitter parent. This sidesteps the competing-conventions problem without any topological analysis. Figure 1 sketches the genotype–phenotype correspondence and the structural mutation operator that the innovation numbering tracks.

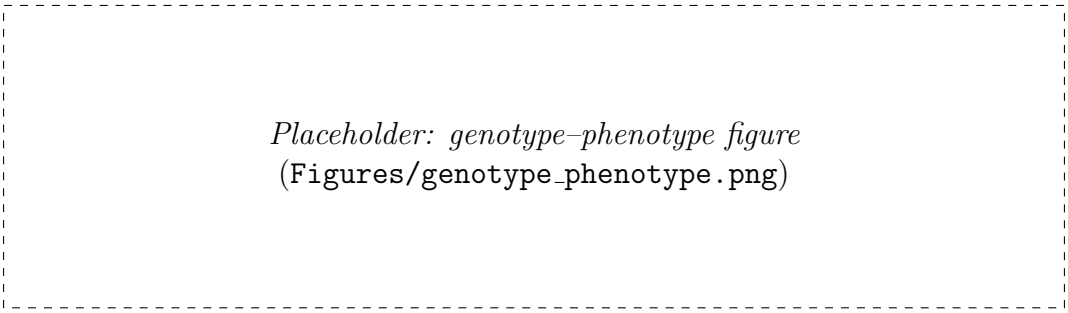


Figure 1: Genotype–phenotype correspondence in NEAT and the structural mutation operator. A node-addition mutation splits an existing connection: the original connection is disabled and two new connections inherit innovation numbers from a global counter.

The second mechanism is speciation. NEAT partitions the population into species by a compatibility distance that counts excess, disjoint, and weight-difference terms between two genomes. Genomes compete mainly within their own species, so a fresh structural innovation has time to be tuned before it faces the full population. The third mechanism is complexification from minimal structure: because the population starts with no hidden nodes, networks

grow only through mutation, which biases the search toward parsimonious topologies without a hand-tuned complexity penalty.

2.4 Activation Functions

An activation function is a nonlinear function applied to each neuron's weighted input. This nonlinearity is necessary for depth to add expressive power: the composition of linear maps is itself linear, so a network without activation functions computes a single linear map regardless of its number of layers (Dubey et al., 2022). The Rectified Linear Unit (ReLU) (Nair & Hinton, 2010) is the canonical example: it outputs zero for negative inputs and the input itself for positive ones.

Common activation functions that are often used are the identity function, the sigmoid function, the hyperbolic tangent, the Rectified Linear Unit (ReLU), and the sine function. These functions differ along the properties commonly used to characterise activation functions: output range, monotonicity, and smoothness (Dubey et al., 2022). The sigmoid and the hyperbolic tangent are the classical choices, both bounded and saturating. The sigmoid maps to $(0, 1)$ and tanh to the zero-centred interval $(-1, 1)$ (Dubey et al., 2022). Their saturation at large inputs later motivated the Rectified Linear Unit, which is piecewise-linear, unbounded above, and cheap to evaluate (Nair & Hinton, 2010). ReLU has since become the default activation in deep learning. In deep convolutional networks such as AlexNet it accelerated training over saturating units (Krizhevsky et al., 2012; Ramachandran et al., 2017). The sine function is uncommon in feedforward training but has a longer history in neuroevolution, where periodic activations let a network encode repeating and symmetric structure compactly (Stanley, 2007). The identity function applies no nonlinearity. A network which uses only identity functions as activation functions collapses to a linear model (Dubey et al., 2022). The choice of activation therefore shapes both which mappings a network can represent and how efficiently it learns them (Dubey et al., 2022). Table 1 summarises the five functions and their properties. Figure 2 plots them.

Artificial neural networks typically use a single activation function across every node (Dubey et al., 2022). That uniformity discards the response diversity of Section 2.1, which lets a heterogeneous population of biological neurons encode roughly twice as much information as a homogeneous one (Padmanabhan & Urban, 2010). Training a network with multiple activation functions, a heterogeneous network, recovers part of this advantage. A survey of

Table 1: Activation functions used in this thesis and their properties.

Function	Formula	Range	Monotone	Smooth
Identity	$f(x) = x$	$(-\infty, \infty)$	yes	yes
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$(0, 1)$	yes	yes
Tanh	$\tanh(x)$	$(-1, 1)$	yes	yes
ReLU	$\text{ReLU}(x) = \max(0, x)$	$[0, \infty)$	yes	no
Sine	$\sin(x)$	$[-1, 1]$	no	yes

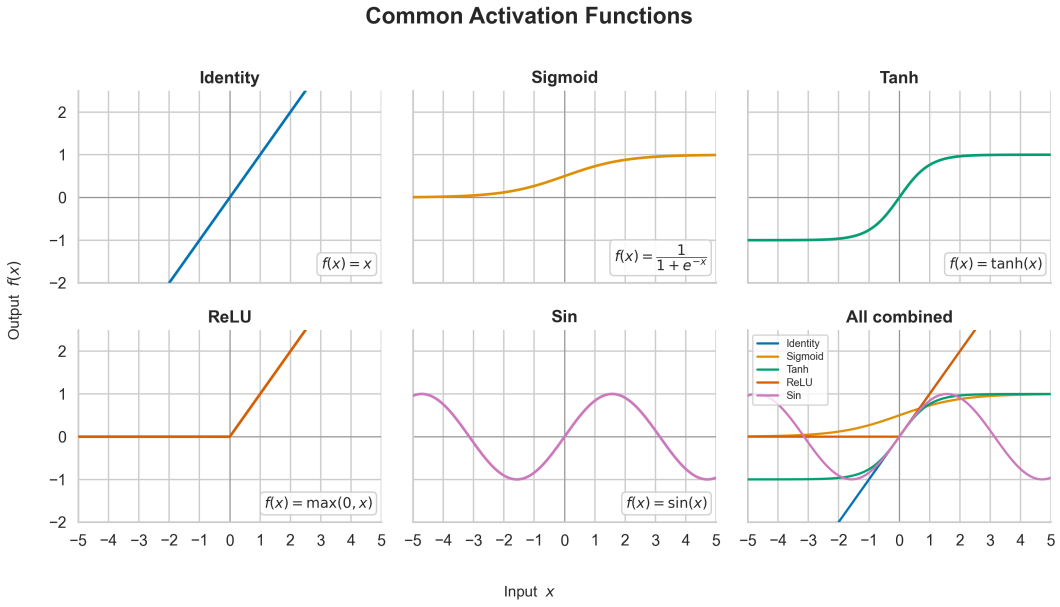


Figure 2: The five activation functions used in this thesis, plotted over the input range. These five constitute the activation set from which HA-NEAT draws each hidden node's activation.

activation functions reports that a mixture outperforms standard ReLU or sigmoid networks across a range of tasks (Dubey et al., 2022). Self-Activating Neural Networks, which use gradient descent to learn a distinct activation per neuron, can in theory represent functions that a fixed-activation network of the same depth cannot (Tutuncuoglu, 2025).

The activation function also shapes the inductive bias of a network, the assumptions a model builds into its hypothesis space before seeing data (Papavasileiou & Jansen, 2017). Papavasileiou and Jansen (2017) showed that the functional form of the activation directly affects the discriminative ability of an evolved network: Gaussian output units in Feature Selective NEAT and Feature Deselective NEAT (FD-NEAT) enabled more accurate feature selection and smaller topologies than sigmoid or ReLU units. Papavasileiou et al. (2022) extended this to HA-FD-NEAT, which evolves heterogeneity, topology, and input selection

jointly, and reported that synchronising a neuron's activation with its connectivity lets evolution match the inductive bias to the task. Functional diversity therefore serves both to compress topology and to optimise input processing. Standard NEAT holds a single activation function fixed across every node, so the diversity these results exploit is unavailable to it. Heterogeneous Activation NEAT (HA-NEAT) makes the activation function itself an evolvable, per-neuron property.

2.5 Heterogeneous Neuroevolution (HA-NEAT)

The HA-NEAT algorithm (Hagg et al., 2017) extends the standard NEAT genome by encoding each neuron's activation function as an additional gene, so that topology, weights, and per-node activation evolve jointly. Its central empirical result is structural parsimony: heterogeneous networks match or exceed the accuracy of homogeneous baselines while using 50%–70% fewer nodes and connections (Hagg et al., 2017). A mutation operator reassigns a node's activation function from a set of qualitatively different forms, such as sine, Gaussian, and ReLU. Swapping an activation changes how the node transforms its input, so the mutation is initially disruptive. HA-NEAT therefore reuses NEAT's speciation mechanism to shield the mutant until its weights co-adapt to the new nonlinearity.

Section 2.4 showed that the activation function also fixes a network's inductive bias. HA-NEAT makes that bias evolvable per neuron, so the reported advantage combines topological compression with a task-adapted functional form (Papavasileiou et al., 2022). Every HA-NEAT result to date was obtained on static classification and regression benchmarks. Whether the same parsimony and inductive-bias advantages transfer to a temporally extended control task remains open. The next two sections set up that question.

2.6 Direct and Indirect Encoding

Neuroevolution methods differ in how the genome maps to a network, and this choice constrains what the algorithm can control. NEAT and HA-NEAT use a direct encoding: every node, connection, and (in HA-NEAT) activation function appears explicitly in the genome, so genotype and phenotype grow together. An indirect encoding severs this correspondence. The main example is HyperNEAT (Stanley et al., 2009), which evolves a Compositional Pattern-Producing Network (CPPN) (Stanley, 2007) that generates each connection weight

from the geometric coordinates of its endpoints. A small CPPN can then specify a network of millions of connections, as long as the task supplies geometric regularity for it to exploit.

We keep the direct encoding, for three reasons. HA-NEAT assigns each neuron its own activation function, and only a genome that lists neurons explicitly can carry that assignment. A CPPN-generated substrate cannot. This thesis also treats network size as a measured quantity under selective pressure, and direct encoding keeps genome size equal to network size, whereas an indirect encoding decouples them. HyperNEAT's own advantages, finally, do not reach this setting. Its compression pays off only for networks far larger than a locomotion controller (Stanley et al., 2009), and its geometric prior fits spatially structured inputs such as a visual field, which the low-dimensional proprioceptive state of Hopper and Walker2D does not supply. Direct encoding thus preserves the exact properties we evaluate, and the open question is the domain in which to evaluate them.

2.7 Continuous-Control Benchmarks

Continuous-control locomotion benchmarks evaluate a policy that outputs real-valued torques for a simulated robot body. The standard benchmark suite (Brockman et al., 2016; Duan et al., 2016) is built on the MuJoCo physics engine (Todorov et al., 2012), whose fast and accurate contact dynamics made physically grounded locomotion tasks tractable for reinforcement-learning research. Two of its standard tasks are used in this thesis. Hopper is a one-legged body that must hop forward without falling. Its state is an 11-dimensional vector of joint positions and velocities, and its action is a 3-dimensional torque vector, one per actuated joint. Walker2D is a planar biped that must walk forward while balancing on two legs. It exposes a 17-dimensional observation and a 6-dimensional action. The two bodies share a kinematic lineage, which makes a single shared controller a plausible target for both tasks. Section 3.1 sets out the overlap in detail.

These tasks are qualitatively harder than the static classification, regression, and low-dimensional toy control settings in which HA-NEAT has historically been validated (Hagg et al., 2017). Three properties account for the difficulty. The action space is high-dimensional and continuous, so the controller outputs real-valued torques instead of a discrete label. Reward arrives only after a sequence of actions, so credit must be assigned across time. And the distribution of states the policy encounters is itself a function of the policy, which makes the learning target non-stationary and unlabelled. A controller that performs well must

therefore encode temporally coherent behaviour over an episode.

Population-based neuroevolution is expensive in this domain because every genome must be rolled out over full episodes to estimate its fitness. Brax (Freeman et al., 2021) reduces this cost by reimplementing MuJoCo-style locomotion in JAX, so that thousands of environment instances run in parallel on a single accelerator and evaluation compiles onto the same device as the search. This shift made population-based evaluation on locomotion tractable at scales that were previously impractical. The search algorithm must match that scale. Classical CPU-bound NEAT libraries do not reach the population sizes and generation counts this study requires. GPU-native implementations such as TensorNEAT (Wang et al., 2024) close the gap by tensorising the evolutionary operators onto the same accelerator that runs the simulation.

Despite the maturity of these benchmarks, heterogeneous-activation neuroevolution has, to our knowledge, never been tested in this domain. Its parsimony and inductive-bias advantages remain established only on the static data of Section 2.5.

2.8 Multi-Task Learning with a Single Shared Network

Multi-task learning is a paradigm where a single model is trained on multiple related tasks simultaneously. By sharing representations and learning processes, the knowledge gained from one task can help improve the performance and generalisation of the others (Y. Zhang & Yang, 2021). Recent empirical analysis of large language models reveals that powerful generalist models naturally develop "task-specific neurons" that specialise in distinct functional roles (Leng & Xiong, 2025). This thesis asks whether providing a set of activation functions lets evolution assign such specialised mathematical roles to specific neurons. We probe the resulting modularity of the control policy only indirectly, through fitness and activation-usage statistics, with no direct measurement of functional specialisation.

To evaluate the efficacy of heterogeneous activation functions in high-dimensional continuous control, this research uses two Brax locomotion environments (Freeman et al., 2021): Hopper and Walker2D. A single evolved network must control both bodies at once using one shared set of weights. The two tasks require diverse motor skills, from single-leg hopping to bipedal balance and forward propulsion. This setting suits the study because the two environments share a kinematic lineage while differing in observation dimensionality and action space. A

single genome must therefore reuse the same circuitry across two interfaces, and specialising in either one leaves it unable to serve the other. While approaches such as Mixture-of-World models address this heterogeneity by explicitly engineering diverse expert modules (B. Zhang et al., 2026), this thesis asks whether a heterogeneous NEAT network can reach similar functional modularity on its own, by evolving a population of neurons tailored to the two locomotion tasks.

2.9 Research Gap

Two lines of work meet in this thesis. Heterogeneous-activation neuroevolution (Section 2.5) shows that letting evolution choose a per-neuron activation function yields smaller networks and a task-adapted inductive bias. Every such result, however, has been established on static, single-task classification data (Section 2.5). Continuous-control locomotion (Section 2.7) is a mature benchmark domain for neuroevolution, yet it has only ever been approached with a homogeneous activation function fixed across all nodes. Whether activation heterogeneity confers an advantage in a temporally extended control setting remains an open question. The same holds for *emergent modularity*: the spontaneous specialisation of neurons within a single shared network that must solve more than one task at once. That intersection is the gap this thesis addresses. Closing it requires three things: an interface that carries two observation and action spaces of different size, an activation set whose members differ in shape, and an evolutionary setup that compares the homogeneous and heterogeneous variants on matched terms. Section 3 specifies each in turn.

3 Methodology

3.1 Problem Formulation

This work addresses the multi-task learning problem of evolving a single neuroevolutionary network capable of controlling two continuous-locomotion agents simultaneously: Hopper and Walker2D. Both originate as MuJoCo control tasks (Brockman et al., 2016; Todorov

et al., 2012) and are used here in their Brax reimplementation (Freeman et al., 2021). The objective is a single shared neural network that receives observations from either environment, generates the appropriate joint-torque actions, and accumulates reward across both tasks, in place of two separately trained controllers. A genome's fitness is therefore defined by its combined performance on both environments, evaluated every generation.

The two environments were introduced in Section 2.7. The dimensions that drive the shared-network design are the observation and action spaces of each environment. Hopper exposes an 11-dimensional observation and a 3-dimensional action, while Walker2D exposes a 17-dimensional observation and a 6-dimensional action. Because these dimensions differ, a single fixed-topology network cannot ingest both environments directly, which motivates the padding scheme described below. The two bodies also overlap anatomically. Hopper is a single planar leg: a torso with thigh, lower-leg, and foot segments connected by three actuated hinge joints. Each of Walker2D's two legs follows the same segment and joint layout as Hopper's single leg, with identical joint axes and ranges. Only foot length and actuator strength differ. Control circuitry evolved for one of these legs can, in principle, drive the other, so the task pair is a suitable setting for a single shared network.

Our hypothesis concerns activation heterogeneity: whether allowing each hidden node to carry its own activation function improves multi-task performance relative to a homogeneous network. We address this question through a controlled comparison of standard NEAT against HA-NEAT in the multi-task setting.

3.2 Genome Representation and the Padding Scheme

Because Hopper and Walker2D differ in observation and action dimensionality, a mechanism is required to present both environments to a single fixed-topology network. The approach adopted in this work is a union padding scheme: the network is constructed with 17 input nodes and 6 output nodes, corresponding to the maximum observation and action dimensions across both tasks. This design was chosen over modular sub-networks to keep the shared-structure argument clean. Any shared structure discovered by evolution arises from optimisation pressure alone, not from an architecture decision.

Formally, given a network $f : \mathbb{R}^{17} \to \mathbb{R}^{6}$, the task-specific action functions are defined as:

$$a_{\text{Hopper}} = f([\mathbf{o}_{\text{Hopper}}; \mathbf{0}_6])_{1:3}$$

$$a_{\text{Walker2D}} = f(\mathbf{o}_{\text{Walker2D}})$$

where $[\cdot;\cdot]$ denotes concatenation and the subscript denotes output slicing. For Hopper, the 11-dimensional observation vector is zero-padded to 17 dimensions before being presented to the network, and only the first 3 of the 6 output dimensions are used as actions. For Walker2D, all inputs and outputs are used directly without modification.

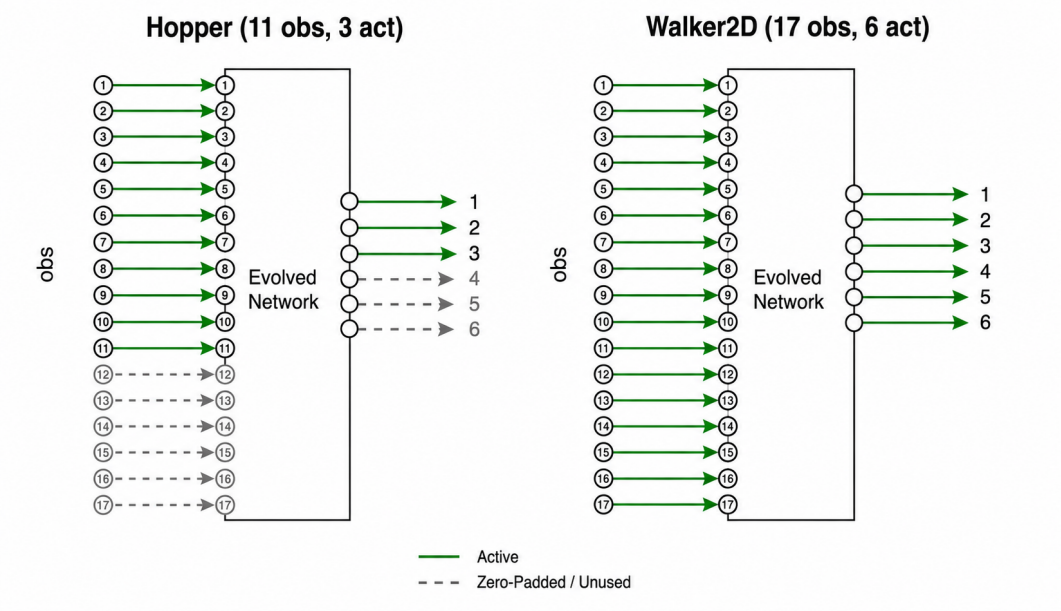


Figure 3: The shared network interface for the multi-task setup. The same evolved network serves both tasks. For Hopper, inputs 12–17 receive zero padding and outputs 4–6 are unused. Walker2D uses all 17 inputs and all 6 outputs natively.

The zero-padded input positions are constant during Hopper episodes, so Hopper applies no selection pressure on the connections leaving them. The network may learn to route Hopper-relevant computation through the first 11 input nodes while reserving the remaining 6 for Walker2D-specific processing, or it may discover shared representations spanning both input regions. Figure 3 illustrates this shared interface.

At the implementation level, the evolved network's nodes are stored as a $[N_{\max} \times 4]$ array encoding index, bias, aggregation function, and activation function per node. Connections are stored as a $[C_{\max} \times 4]$ array encoding source index, destination index, historical marker, and weight per connection. Inactive positions carry NaN in the first column of either array. A gene is active exactly when its first entry is not NaN.

3.3 Algorithm Design: HA-NEAT

Section 2.5 motivated heterogeneous activations and the speciation protection that shields them. This section specifies the single algorithm, HA-NEAT, that provides both.

3.3.1 NEAT Foundation

NEAT evolves network topology and weights jointly through historical marking, speciation, and complexification from minimal structure, as introduced in Section 2.3. We restate here only the details the HA-NEAT extensions build on.

Each genome encodes a variable-topology neural network as two gene lists: node genes and connection genes. A node gene stores a unique index, a bias value, an aggregation function that combines the node's incoming signals into one value (Section 3.5), and an activation function. A connection gene stores source and destination node indices, a historical marker, and a weight value. The array-level storage of these gene lists is described in Section 3.2.

The population is partitioned into species based on genetic distance. Two connection genes are *homologous*, that is of shared ancestry, when they carry the same historical marker, meaning both descend from the same ancestral structural mutation. A gene whose marker appears in only one genome is non-homologous. For each pair of genomes, the distance function aligns connection genes by their identifying attributes, counts non-homologous genes, and computes attribute differences on homologous genes. Where the encoding carries a historical marker, the identifying attributes are that marker together with the (source, destination) index pair. Otherwise they are the index pair alone. Section 4.2 contrasts these two ways of identifying a connection gene, by marker or by index pair. The distance between two genomes is

$$d = \frac{D \cdot c_d + \sum_{i \in \mathcal{H}} \Delta_i \cdot c_w}{\max(n_1, n_2)}$$

where D denotes the count of non-homologous genes, $\mathcal{H}$ denotes the set of homologous gene pairs, Δ_i is the attribute distance for pair i, c_d and c_w weight the two terms (values in Table 3), and n_1, n_2 are the gene counts of each genome. The distance is computed separately for node genes (aligned by node index) and for connection genes (aligned by their

identifying attributes). The two values are then summed. For a homologous connection pair, Δ_i is the absolute weight difference $|w_1 - w_2|$. For a homologous node pair it is

$$\Delta_i = |b_1 - b_2| + \mathbb{1}[\text{agg}_1 \neq \text{agg}_2] + \mathbb{1}[\text{act}_1 \neq \text{act}_2],$$

the absolute bias difference plus one unit for each mismatched aggregation or activation function. An activation mutation therefore raises the genetic distance twice over: directly, through this activation term, and indirectly, through the marker reassignment of Section 3.3.2. This form follows the TensorNEAT (Wang et al., 2024) implementation and merges the separate disjoint and excess terms of the original NEAT formulation into the single count D. It also divides both terms by $\max(n_1, n_2)$, whereas the original formulation averages attribute differences over homologous pairs only. Under this definition, attribute differences therefore contribute less to the distance in large genomes than they do under the original one. Genomes whose total distance falls below the compatibility threshold θ (Table 3) are assigned to the same species.

Within each species, the top fraction of genomes, set by the survival threshold (Table 3), produces offspring through crossover and mutation. During crossover, genes are aligned by historical marker. Homologous genes are inherited randomly from either parent, and non-homologous genes come from the fitter parent.

3.3.2 HA-NEAT Extensions

HA-NEAT extends the standard NEAT algorithm with three modifications, all encapsulated within the mutation operator. The speciation-driven evolutionary loop, crossover, and distance computation remain unchanged. Each modification is described in turn.

1. **Random activation assignment upon node creation.** In standard NEAT, a newly added node receives the default activation function (tanh). HA-NEAT instead assigns a random activation function drawn uniformly from $\mathcal{A} = \{\tanh, \sigma, \text{ReLU}, \sin, \text{identity}\}$ at node creation. This introduces immediate functional diversity: the new node contributes a non-trivial transformation from its first generation, avoiding the evolutionary delay of waiting for a separate activation mutation.

2. **Controlled activation mutation with speciation protection.** With probability

p_{act} (Table 3) per genome per generation, exactly one randomly selected hidden node has its activation function replaced by a function selected from $\mathcal{A}$ with equal probability across all elements of the set. Upon this mutation, all connections attached to the mutated node receive new historical markers from a pre-allocated pool, increasing the genetic distance between the mutated genome and its relatives. This causes the speciation mechanism to place the mutant into a separate species, protecting it from competitive elimination before its weights can co-adapt to the new activation landscape. Section 3.6 details the pre-allocated marker pool and the compilation constraint that motivates it.

3. **Suppression of standard activation mutation.** Standard NEAT can also change a node's activation function as a side effect of weight and bias mutation, with its own per-node probability. HA-NEAT switches this off so that activations change only through the controlled mechanism above. In practice, it saves the activation column before weight and bias mutation and restores it afterward, so that mutation cannot alter any activation. This matters because such a change would carry no marker reassignment and therefore no speciation protection, exposing the disrupted genome to immediate competitive pressure.

3.4 Fitness Evaluation

A critical design decision in multi-task neuroevolution is how to aggregate performance across tasks into a single selection signal. Each generation, every genome in the population is evaluated on both Hopper and Walker2D through the following pipeline:

1. **Decode.** The genome's node and connection arrays are transformed into an executable network representation via topological sort and adjacency matrix construction.

2. **Evaluate per task.** The network controls each environment for 1000 timesteps. Each task draws its randomness from its own JAX pseudo-random-number key, derived deterministically from the genome's evaluation key, so the same seed always reproduces the same episodes.

3. **Normalise.** Each raw per-task return is divided by a fixed task-specific constant that accounts for the differing reward scales across environments: 3000 for Hopper and 5000 for Walker2D. We set each constant near a strong single-task return for its

environment, so that a normalised score of 1.0 marks competence comparable to a well-trained deep reinforcement learning agent. The two values track published reference points closely. The Deep Data-Driven Reinforcement Learning (D4RL) normalisation scores, defined from a trained Soft Actor-Critic (SAC) policy, are 3234 for Hopper and 4592 for Walker2D (Fu et al., 2020; Haarnoja et al., 2018), and reported maximum returns over one million steps reach about 3564 and 4683 for TD3 (Fujimoto et al., 2018). These figures come from the MuJoCo versions of the tasks, so we treat them as approximate scale references and not as exact targets.

4. **Aggregate via the normalised minimum.** The two normalised per-task scores are collapsed into a scalar by taking their minimum.

Formally, writing r_{Hopper} and r_{Walker2D} for the raw per-task returns, the fitness of a genome is

$$\text{fit} = \min\left(\frac{r_{\text{Hopper}}}{3000},\ \frac{r_{\text{Walker2D}}}{5000}\right).$$

The choice of the minimum follows one principle: an aggregation must not reward single-task specialisation. Walker2D's reward scale substantially exceeds Hopper's, and a sum of the normalised scores allows a genome to attain a high aggregate value by excelling at one task while neglecting the other. A product drives the aggregate toward zero only if one score collapses entirely. Short of that it still permits substantial imbalance, because a strong score on one task multiplicatively compensates for a weak but nonzero score on the other. The minimum ties a genome's fitness to its *worse* task, so a network earns reward only once it is competent on both environments at once. Our argument for the minimum is therefore structural: of the three aggregators, it alone cannot reward a genome for neglecting a task. We defer the empirical support to the aggregation calibration experiment (Section 4.4), which confirms both weaknesses. Sum and product aggregation there produced genomes that scored well on the aggregate while neglecting one task, an outcome the minimum excludes by construction.

3.5 Network Architecture

All networks are initialised with a minimal topology: 17 input nodes connected directly to 6 output nodes, with no hidden nodes. This follows the NEAT principle of complexification: structural complexity emerges through mutation and selection pressure rather than through designer assumptions about required network depth or width.

Output nodes apply a tanh activation to bound their values within $[-1, 1]$. This constraint is necessary because Brax environments interpret actions as normalised joint torques: each action dimension specifies the magnitude and direction of force applied to a joint actuator, bounded to the unit interval.

Hidden nodes use per-node activation functions. In the standard NEAT condition, all hidden nodes use tanh. In the HA-NEAT condition, the activation set $\mathcal{A} = \{\tanh, \sigma, \text{ReLU}, \sin, \text{identity}\}$ enables heterogeneous internal representations. The individual activation functions were selected for their complementary computational properties:

- **Tanh and sigmoid (σ)**: Bounded and smooth, these functions are suitable for gating behaviour and saturation dynamics, where network outputs must be constrained within a fixed range.
- **ReLU**: One-sided and non-saturating, unbounded in the positive domain (Nair & Hinton, 2010). In the small evolved topologies of this study it supplies rectified, piecewise-linear control signals. The depth-related benefits reported for deep architectures do not apply at this scale.
- **Sin**: Periodic by nature, sinusoidal activation is capable of encoding oscillatory patterns, a property of particular relevance to rhythmic locomotion, where joint trajectories follow approximately periodic profiles.
- **Identity**: A linear pass-through that enables skip connections and linear sub-networks, allowing the evolved topology to incorporate proportional control pathways without nonlinear distortion.

All hidden nodes employ sum aggregation. That is, the pre-activation value of a node is the sum of incoming weighted signals plus a bias term, and the node's output is that value passed

through the node's activation function. Formally, for node j with incoming connections from the set of source nodes $\mathcal{I}_j$:

$$z_j = b_j + \sum_{i \in \mathcal{I}_j} w_{ij} \cdot v_i, \qquad v_j = \phi_j(z_j)$$

where z_j is the pre-activation value, v_j the output of node j, $\phi_j \in \mathcal{A}$ denotes the node's activation function, b_j is the bias, w_{ij} are connection weights, and v_i are the output values of source nodes.

3.6 Implementation Stack

The experiments are built upon TensorNEAT (Wang et al., 2024), a GPU-accelerated NEAT implementation constructed on JAX (Bradbury et al., 2018), a Python library for accelerator-based array computation. TensorNEAT encodes the entire evolutionary pipeline (population initialisation, mutation, crossover, speciation, fitness evaluation, and selection) as JAX-transformable functions. These compile to a single XLA computation graph through `jax.jit`, which translates the Python functions once into an optimised GPU program, and the graph then runs entirely on the GPU without CPU-GPU data transfers during the evolutionary loop. Population-level operations are vectorised using `jax.vmap`, which applies the same function across the whole population in parallel instead of looping over genomes in Python. This end-to-end GPU execution was the primary motivation for selecting TensorNEAT over classical CPU-based NEAT implementations such as NEAT-Python.

JAX's static-shape requirement also constrains the HA-NEAT mutation operator (Section 3.3.2). The mutation operator pre-allocates $3 + C_{\max}$ historical markers per genome per generation, where $C_{\max}$ denotes the maximum number of connections: the first three serve the standard structural mutations, and the remaining $C_{\max}$ cover the worst case of an activation mutation that re-marks every connection incident to a single node. Pre-allocation keeps marker assignment free of dynamic memory allocation inside the compiled graph.

Brax (Freeman et al., 2021) provides the physics backend, implementing rigid-body simulation as differentiable JAX functions that integrate directly with TensorNEAT's JIT-compiled evaluation loop. The combination of JAX-native evolution and JAX-native simulation eliminates the host-device transfer bottleneck that limits frameworks in which the evolutionary

algorithm runs on the CPU while fitness evaluation runs on the GPU.

Experiment tracking is implemented using MLflow (Zaharia et al., 2018), an open-source platform for logging and comparing machine learning experiments. Each run logs per-generation metrics including maximum, mean, minimum, and standard deviation of fitness across the population, alongside valid genome count, generation wall-clock time, species count, stagnation counter, and the node and connection counts of the best genome. Per-task fitness scores for the best genome are tracked separately for Hopper and Walker2D, enabling post-hoc analysis of task-specific performance trajectories and the degree to which the shared network specialises toward one environment over the other. The aggregated `best_fitness` metric records the normalised minimum defined in Section 3.4, so its per-generation trajectory tracks the worse of the two normalised per-task rewards.

4 Experimental Settings

4.1 Environment Details

The two environments share an episode structure but differ in body and interface size, as summarised in Table 2.

Table 2: Specifications of the two Brax evaluation environments, including the zero-padded interface dimensions the shared network presents to each task.

Property	**Hopper**	**Walker2D**
Observation dimensions	11	17
Action dimensions	3	6
Padded input dimensions	17 (6 zeros appended)	17 (native)
Padded output dimensions	6 (last 3 unused)	6 (native)
Episode length	1000 timesteps	1000 timesteps
Action range	[-1, 1] (continuous)	[-1, 1] (continuous)
Simulator	Brax v0.14.1 (MJX, MuJoCo on GPU)	Brax v0.14.1 (MJX)

Both environments reward forward locomotion while penalising energy expenditure and falls. Hopper terminates an episode early if the torso height falls below a threshold or the torso tilts

beyond a critical angle. Walker2D applies analogous termination conditions for its bipedal stance. Both follow the original MuJoCo task definitions (Brockman et al., 2016; Todorov et al., 2012) as reimplemented in Brax (Freeman et al., 2021). The reward structures differ in scale: Walker2D typically yields higher cumulative rewards than Hopper for equivalent locomotion quality. This scale asymmetry motivates the per-task normalisation constants (3000 for Hopper, 5000 for Walker2D) and the normalised-minimum aggregation defined in Section 3.4.

4.2 Experimental Conditions

The experiment compares three conditions, all operating within the multi-task Hopper + Walker2D setting:

1. **Multi-Task NEAT (baseline).** Standard NEAT with homogeneous tanh activation functions on all hidden nodes. This condition uses `DefaultConn`, which carries no historical markers. Gene identity is instead given by the (source, destination) index pair, so crossover and distance align connection genes by structure rather than by marker. Absent marker reassignment the two identity schemes agree almost everywhere, since a connection's endpoints never change after creation. They diverge in one case only: when two lineages independently create the same connection, the marker scheme counts two distinct genes and the index-pair scheme counts one. The standard per-node activation mutation mechanism is active but has no practical effect, since only one activation option is available.

2. **Multi-Task HA-NEAT (treatment).** HA-NEAT with heterogeneous activations drawn from the activation set $\mathcal{A}$. This condition uses `OriginConn` with historical markers. Activation mutation occurs at most once per genome per generation, with speciation protection via marker reassignment as described in Section 3.3.2.

3. **HA-NEAT ablation (tanh only).** The full HA-NEAT algorithm (`OriginConn`, controlled activation mutation, and marker reassignment) restricted to a single activation function, tanh. Activation mutation still fires with probability p_{act} per genome per generation. With a single option available the activation change itself has no functional effect, but the accompanying marker reassignment still occurs. This condition isolates HA-NEAT's structural mechanisms from its activation diversity: it tests whether an

advantage of the treatment requires heterogeneity itself, or whether the surrounding speciation mechanism already produces that advantage.

All three conditions use identical population sizes, generation limits, structural mutation rates, and evaluation procedures (Table 3). The independent variable is the activation function regime: homogeneous activations, heterogeneous activations with speciation protection, and the ablation that retains the speciation mechanism without heterogeneity. The dependent variables are:

- **Per-task performance**: Each champion's normalised return on Hopper and Walker2D individually, taken as a seed-level mean over the 20 re-evaluation episodes per task, and used to identify which task binds the normalised minimum, that is, which environment limits a genome's aggregate fitness.
- **Genome complexity**: The number of active nodes and connections in the champion, the structural cost of the evolved solution.
- **Species dynamics**: The number of active species, a proxy for population diversity.
- **Activation composition** (HA-NEAT only): The distribution of activation functions among the hidden nodes of the final champions, measuring whether the evolved networks exercise the heterogeneity the treatment provides.

4.3 Hyperparameters

Table 3 lists the evolutionary hyperparameters used across the three experimental conditions. The values are taken from the run parameters logged in MLflow and are identical across all 90 runs of the main study.

The compatibility threshold was carried over as $\theta = 0.3$ from earlier large-population experiments, which had compared thresholds of 0.3 and 0.5. At population 200 its exact value made little difference. The setting filled the configured maximum of 10 species in every condition (Section 5.3), so the species count stayed at this ceiling rather than responding to the threshold, which normally governs how many species form. The survival threshold dictates that the top 30% of each species produces offspring, creating within-species selection pressure while speciation protects structural diversity across the population as a whole.

The fitness target of 10,000 lies far outside the attainable range of the normalised-minimum objective, so no run stopped early. Every run completed the full 750 generations.[1]

4.4 Preliminary Calibration Experiments

Two preliminary experiments calibrated the design of the main study before its full seed grid was run. The first, the aggregation calibration experiment, compared the three candidate ways of collapsing the two normalised per-task scores into a scalar: their sum, their product, and their minimum. The grid covered two algorithms, three population sizes, and two generation budgets at a single seed, yielding twelve champions per aggregation. Sum and product permit a genome to score well on the aggregate while neglecting one task. The most extreme case reached a normalised Hopper score of 1.04 while its Walker2D score collapsed to 0.17 (Figure 4). Consistent with this structural weakness, the minimum produced the smallest mean gap between a genome's stronger and weaker task (0.11, against 0.20 for sum and 0.18 for product) and the highest average weaker-task score. Because the grid used a single seed we read these differences as a tendency. The main reason for choosing the minimum is structural. The minimum cannot reward specialisation, and it was therefore adopted as the selection signal defined in Section 3.4. The second, the training-budget calibration runs, evolved larger populations (1024 to 4096 genomes) for up to 2000 generations and tracked how far best fitness continued to improve. These runs reached on average 91–93% of their eventual 2000-generation best by generation 750, and the improvement rate per 100 generations fell four- to five-fold after generation 700 (Figure 5). A 750-generation budget therefore captures the large majority of attainable fitness within the diminishing-returns regime, while making the 30-seed, three-condition design of the main study computationally tractable. The main-study runs at population 200 decelerate even earlier than the calibration runs, so the budget transfers across the population-size difference.

4.5 Evaluation Protocol

Each experimental condition was repeated across 30 independent runs. All three conditions used the same 30 random seeds. The seed list, the full run configuration, and the

[1] One NEAT run (seed 5213) was aborted at generation 324 and re-run to completion. The truncated run is excluded from all analyses.

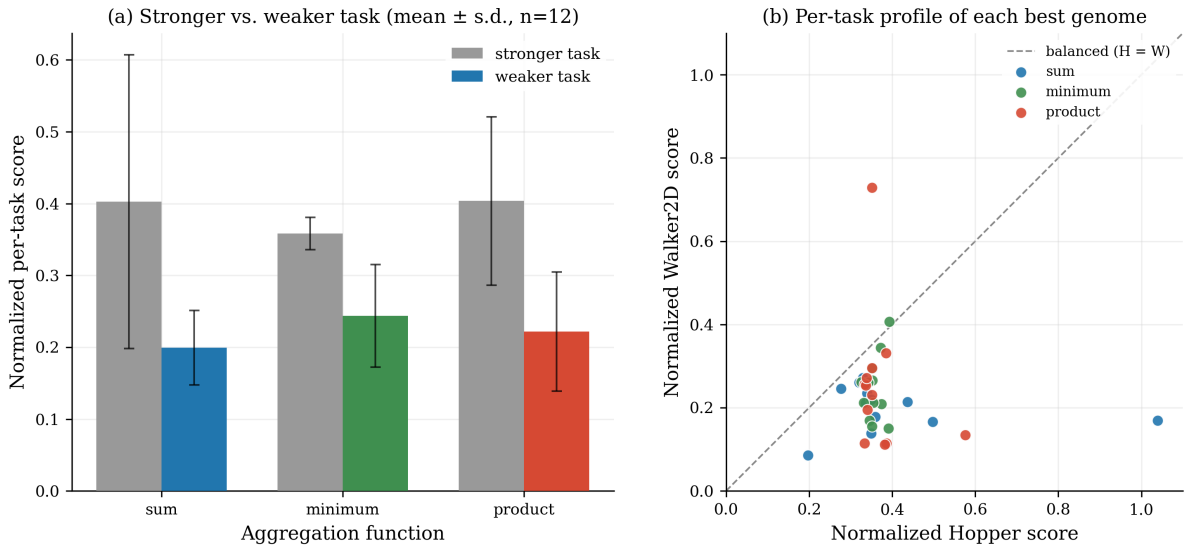


Figure 4: Aggregation calibration experiment (twelve champions per aggregation, single-seed grid). (a) Mean normalised score of each genome's stronger and weaker task under sum, minimum, and product aggregation. The minimum yields the smallest stronger–weaker gap and the highest weaker-task score. (b) Per-task profile of each champion. The dashed diagonal marks balanced performance. Sum and product permit strongly unbalanced genomes, including one sum-aggregation genome with a normalised Hopper score of 1.04 and a Walker2D score of 0.17.

analysis code are available on GitHub.[2] At $\alpha = 0.05$ and 80% power, meaning an 80% chance of detecting a real effect, this sample size can detect effects of roughly Cohen's $d \geq 0.74$ with non-parametric tests, while remaining computationally tractable given the cost of population-based neuroevolution on GPU hardware.

Performance is measured at three levels:

1. **Per-generation tracking.** The training pipeline logs the dependent variables of Section 4.2 every generation, alongside population-level fitness statistics (maximum, mean, minimum, and standard deviation). The exception is activation composition, which we compute once from the saved champion genomes.

2. **Final performance.** The champion of each run was re-evaluated after training over 20 episodes per task with fresh random seeds, and the run's score is the normalised minimum of the two per-task re-evaluation means. Training fitness rests on a single episode and is therefore a noisy estimate of policy quality. The re-evaluation removes this noise from the between-condition comparison. Scores are reported as median and

[2] https://github.com/SvenGoerdes/tensorneat

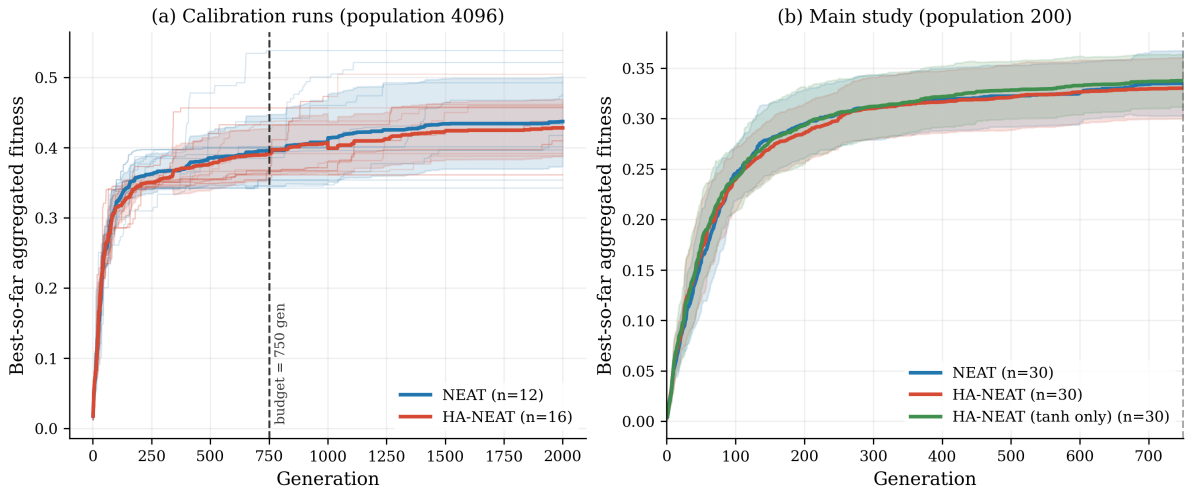


Figure 5: Training-budget calibration. (a) Best-so-far normalised-minimum fitness against generation for the calibration runs (population 4096, up to 2000 generations). The dashed vertical line marks generation 750. (b) The same curves for the main study (population 200, 750 generations). Improvement rates fall four- to five-fold after generation 700 in every condition, and the smaller main-study populations decelerate earlier than the calibration runs, supporting the 750-generation budget.

interquartile range (IQR) across the 30 independent runs. Each run contributes a single value, and these per-run values are the data points for all statistical tests.

3. **Learning dynamics.** For each condition we average the fitness-over-generation curve across its runs and add a 95% confidence band, which shows how fast and how stably the condition converges.

Statistical comparison across the three conditions uses a Kruskal–Wallis omnibus test ($\alpha = 0.05$) on the seed-level re-evaluation scores, followed by Holm-corrected pairwise Mann–Whitney tests. Non-parametric tests are appropriate because evolutionary run outcomes are not guaranteed to follow a normal distribution and do not reliably support parametric assumptions. Each pairwise contrast is accompanied by a Hodges–Lehmann shift estimate with a bootstrap 95% confidence interval and by Cliff's δ as an effect size, and equivalence testing by Welch two one-sided tests (TOST) complements the significance tests where the question is whether conditions perform the same. The omnibus test is reported with epsilon-squared, $\varepsilon^2 = (H - k + 1)/(n - k)$, the proportion of rank variance attributable to condition, where k is the number of conditions and n the total number of runs. Values near 0 indicate no effect.

4.6 Compute Resources

All experiments were conducted on a single compute node with the specifications listed in Table 4.

Each evolutionary run utilises a single GPU. The dual-GPU configuration enables two independent runs to execute in parallel, halving the wall-clock time required for the full experimental campaign. Across the 90 main-study runs, a single run took a median of roughly 3.4 h (mean 3.8 h), for a campaign total of about 340 GPU-hours. JIT compilation of the evolutionary step occurs once at the start of each run. Subsequent generations execute the compiled XLA graph without recompilation overhead, with the exception of topology-triggered recompilation events that occur when the population's maximum genome size changes.

5 Results and Discussion

5.1 Experimental overview

We evaluated NEAT and HA-NEAT on a single multi-task control problem: one evolved network must control both the Hopper body (11 observations, 3 actions) and the Walker2D body (17 observations, 6 actions) using the same weights, through the zero-padding and output-slicing interface of Section 3.2. The main study compared three conditions: standard NEAT (tanh only), HA-NEAT (per-node activations drawn from tanh, sigmoid, relu, sin, identity), and an ablation running HA-NEAT's full algorithm restricted to tanh. Each condition was evolved at population 200 for 750 generations across 30 independent random seeds, giving 90 runs in total on the MJX physics backend.

The selection signal was the normalised minimum of the two per-task scores, so a genome cannot raise its score by specialising in one body and neglecting the other. We motivate this objective in Section 3.4. The calibration study (Section 4.4) justified it empirically against the sum and product alternatives and fixed the 750-generation budget. Preliminary NEAT-versus-HA-NEAT comparisons shaped the rest of the design. We ran them at populations of 1024–4096 with only two to three seeds per condition. They compared the compatibility thresholds carried into Section 4.3, and their failure to separate the two algorithms at that seed count motivated the 30-seed design and the tanh-only ablation that isolates the mutation mechanism from activation diversity. These preliminary runs only shaped the design; the main study reported here replaces them as the reported result.

Section 5.2 reports the head-to-head multi-task fitness across the three conditions, which is the study's primary result. Section 5.3 examines genome complexity and generational dynamics, Section 5.4 the per-task trade-off and multi-task pressure, and Section 5.5 activation-function usage within HA-NEAT. Section 5.6 discusses these analyses together.

5.2 Multi-task fitness across conditions

For each of the 90 runs we took the champion, re-evaluated it over 20 episodes per task, and computed its normalised-minimum fitness from the two per-task re-evaluation means, giving $n = 30$ seed-level values per condition. This re-evaluation is the reference performance metric for the study.

The three conditions performed near-identically (Table 5). NEAT reached a median fitness of 0.184 (IQR $[0.124, 0.253]$), HA-NEAT 0.208 ($[0.152, 0.276]$), and the ablation 0.199 ($[0.152, 0.258]$). A Kruskal–Wallis test found no difference across the three conditions ($H = 1.48$, $p = 0.476$, $\varepsilon^2 \approx 0$). Pairwise, every contrast was small and every confidence interval included zero. The Hodges–Lehmann shifts, their bootstrap intervals, and Cliff's δ for all three contrasts are reported in Table 5. For readers calibrated to Brax's raw reward scale: because Walker2D sets the minimum for most champions (Section 5.4), a normalised-minimum fitness of 0.20 corresponds to a Walker2D return of roughly 1000, against the 5000 used for normalisation.

The design bounds how large an undetected effect could be. With $n = 30$ per group, the smallest difference detectable at $\alpha = 0.05$ (two-sided) with power 0.80, adjusted for the asymptotic relative efficiency of the Mann–Whitney test, is Cohen's $d \approx 0.74$, about 0.06 normalised-fitness units or roughly 30% of the observed performance level. We can therefore rule out large differences between conditions. Smaller differences could have gone undetected. The two one-sided tests (TOST) make this more precise, although the procedure is parametric and therefore serves only as a supplementary check alongside the rank-based analyses: at an equivalence bound of ± 0.8 pooled SD (± 0.067) all three contrasts are statistically equivalent ($p = 0.034$, 0.020, 0.002), whereas at the tighter ± 0.5 SD bound (± 0.042) only HA-NEAT versus the ablation is equivalent ($p = 0.040$), with NEAT versus HA-NEAT ($p = 0.234$) and NEAT versus the ablation ($p = 0.176$) inconclusive. The study therefore rules out large (≥ 0.8 SD) differences everywhere, and medium differences only between HA-NEAT and its ablation. The confidence intervals and the minimum detectable effect are the primary evidence for the null, with the TOST results supporting equivalence only at the large-effect bound.

The no-difference result holds across the whole training run, and not only at the final score (Figure 6). All three conditions traced the same learning curve: a steep rise over the first 150 generations, then a slow climb through the diminishing-returns regime that the budget

calibration anticipated (Section 4.4). At no generation did the across-seed means separate by more than 0.03 normalised-fitness units, and the 95% confidence bands overlap throughout: the curves stay interleaved from start to finish. Over the final 50 generations the mean training fitness was 0.301 (NEAT), 0.307 (HA-NEAT), and 0.318 (ablation). These training values exceed the re-evaluated scores of Table 5 because training fitness rests on a single episode per genome and therefore rewards lucky rollouts (Section 4.5).

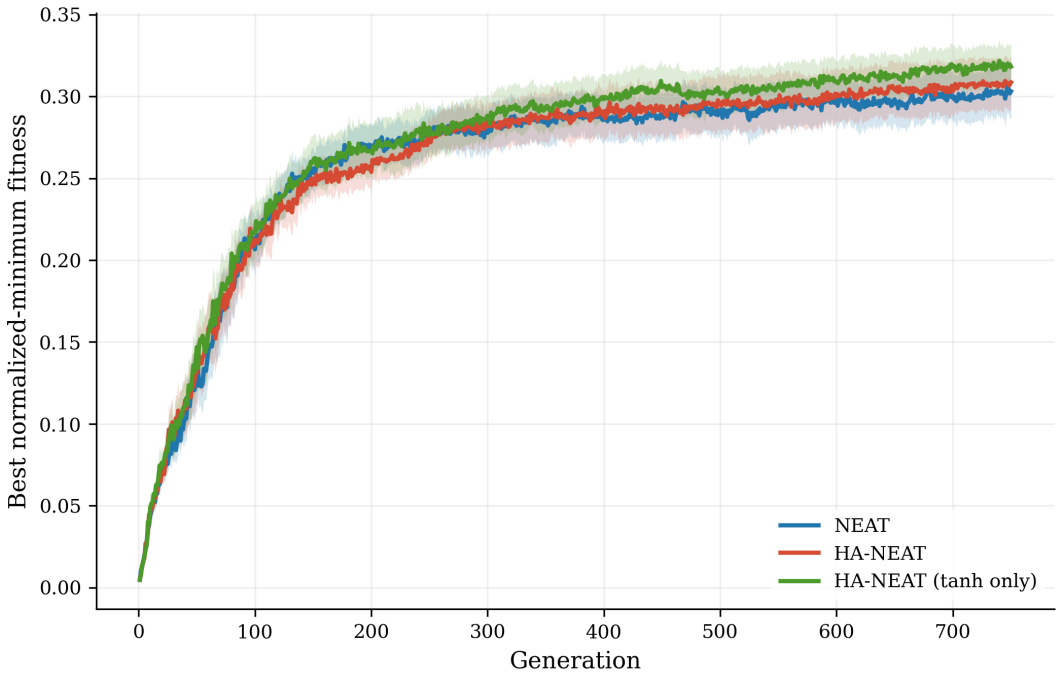


Figure 6: Learning dynamics in the main study. Lines are across-seed means ($n = 30$ per condition) of the per-generation best normalised-minimum fitness. Shaded bands are 95% confidence intervals of the mean. The three conditions follow the same trajectory over all 750 generations, and their means never separate by more than 0.03 normalised units.

Table 3: Evolutionary hyperparameters, identical across all three conditions except for the HA-NEAT-specific block, which applies only to HA-NEAT and its ablation.

Parameter	**Value**
Population size	200
Generation limit	750
Fitness target	10,000 (raw return, unnormalised)
Max nodes per genome	50
Max connections per genome	200
Maximum species count	10
Compatibility threshold (θ)	0.3
Distance coefficients (c_d / c_w)	1.0 / 0.4
Survival threshold	0.3 (fraction of each species)
Max species stagnation	40 generations
Species elitism	2
Genome elitism	2
Connection add probability	0.2
Connection delete probability	0.2
Node add probability	0.1
Node delete probability	0.1
Bias init mean / std	0.0 / 1.0
Bias mutate power / rate	0.15 / 0.2
Bias replace rate	0.015
Weight init mean / std	0.0 / 1.0
Weight mutate power / rate	0.15 / 0.2
Weight replace rate	0.015
HA-NEAT specific	
Activation mutation rate (p_{act})	0.1 per genome per generation
Activation set	$\{\tanh, \sigma, \text{ReLU}, \sin, \text{identity}\}$
Standard activation replace rate	0.0 (disabled)
Historical marker reassignment	Enabled (via OriginConn)

Table 4: Hardware and software specification of the single compute node on which all 90 main-study runs were executed.

Component	**Specification**
CPU	AMD Threadripper 7970X (32 cores, 64 threads)
RAM	251 GB
GPU	2× NVIDIA RTX A6000 (48 GB VRAM each)
CUDA	12.4
Software	JAX 0.9.1, Brax 0.14.1, Python 3.11

Table 5: Best-found normalised-minimum fitness per condition ($n = 30$ each), from champions re-evaluated over 20 episodes per task. Pairwise rows report the Hodges–Lehmann median-difference estimate with a bootstrap 95% confidence interval and Cliff's δ with its 95% interval. No contrast is significant.

Condition	Median [IQR]	Mean ± SD
NEAT	$0.184\ [0.124, 0.253]$	0.188 ± 0.088
HA-NEAT	$0.208\ [0.152, 0.276]$	0.214 ± 0.084
Ablation	$0.199\ [0.152, 0.258]$	0.210 ± 0.080
Contrast	**Hodges–Lehmann shift [95% CI]**	**Cliff's δ [95% CI]**
NEAT vs. HA-NEAT	$-0.027\ [-0.097, +0.054]$	$-0.18\ [-0.46, +0.11]$
NEAT vs. ablation	$-0.023\ [-0.071, +0.044]$	$-0.12\ [-0.42, +0.17]$
HA-NEAT vs. ablation	$+0.006\ [-0.051, +0.063]$	$+0.04\ [-0.26, +0.34]$

Statistical power is only one part of the explanation for this null. Section 5.6 develops a mechanistic account alongside it: under the minimum objective, once Hopper is kept alive, selection presses almost entirely on improving Walker2D, so the effective problem degenerates toward "improve Walker2D subject to keeping Hopper viable." If the multi-task pressure under-exercises the activation-diversity lever in this way, comparable fitness across conditions is the expected outcome, and the null carries information beyond limited power.

We ran the analyses that follow *after* this null result. They ask a second question: whether conditions that reach equal fitness do so with equivalent internal structure and task balance. These are exploratory, post-hoc analyses, and we frame them as such throughout.

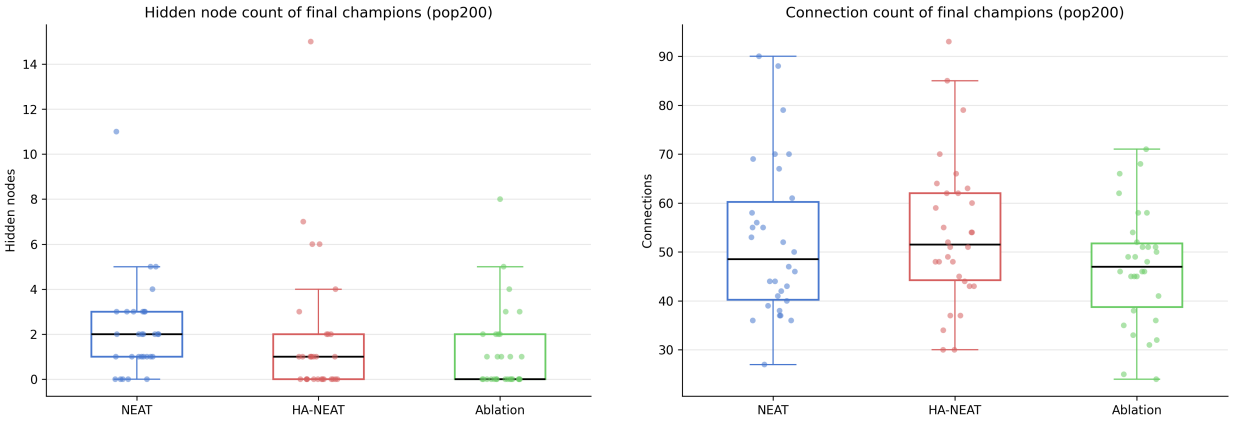
5.3 Genome complexity and generational dynamics

Because the three conditions reached statistically indistinguishable fitness (Section 5.2), we examined whether they arrived there with equivalent genome structure. We ask whether conditions that reached the same fitness did so with the same structure, or whether the HA-NEAT algorithm leaves a topological signature the fitness comparison alone would miss. For each of the 90 champion genomes (30 per condition) we counted hidden nodes and active connections from the raw genome arrays. The 17 input and 6 output nodes are fixed by the task and excluded from the hidden-node count.

The number of hidden nodes in the final champions differed across conditions (Figure 7a). NEAT champions carried a median of 2.0 hidden nodes (IQR $[1.0, 3.0]$), HA-NEAT 1.0 ($[0.0, 2.0]$), and the ablation 0.0 ($[0.0, 2.0]$). A Kruskal–Wallis test rejected equality of the three distributions ($H = 7.45$, $p = 0.024$, $\varepsilon^2 = 0.063$). Holm-corrected pairwise Mann–Whitney tests localised the effect to a single contrast: NEAT versus the ablation ($p = 0.030$, Cliff's $\delta = 0.38$, medium), while NEAT versus HA-NEAT ($p = 0.10$) and HA-NEAT versus the ablation ($p = 0.52$) were not significant. Connection counts did not differ across conditions (medians 48.5 / 51.5 / 47.0; $H = 2.82$, $p = 0.244$; Figure 7b).

The per-generation trajectories from the training logs locate when this gap opens (Figure 8a). Best-genome node counts overlapped across all three conditions until roughly generation 300, after which NEAT rose above both HA-NEAT variants, reaching a final-window mean of 26.6 total nodes (including the 23 fixed input/output nodes) against 24.7 (HA-NEAT) and 24.0 (ablation). Connection counts pruned in lockstep across all three conditions, falling from

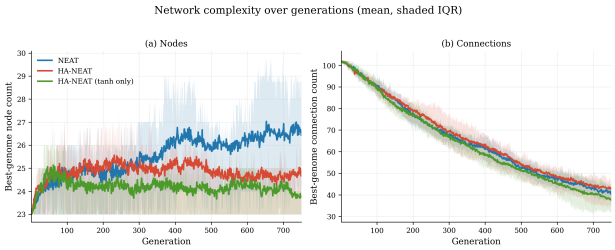
roughly 100 to 40–44 by the final window. Species counts saturated near the same ceiling in every condition (final-window means 9.97 / 9.97 / 9.97; Kruskal–Wallis $H = 0.11$, $p = 0.945$; Figure 8b), pinned at the configured maximum of 10 species. The conditions therefore converged to equal fitness while ending at different topologies: the ablation and HA-NEAT settled on more compact networks than NEAT.



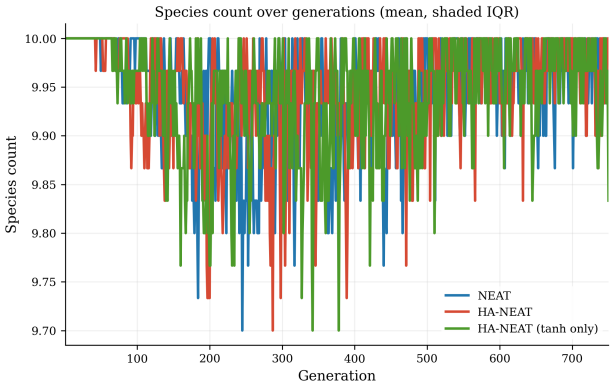
(a) Hidden nodes per champion.

(b) Active connections per champion.

Figure 7: Final-champion complexity in the main study ($n = 30$ per condition). (a) Hidden-node distributions differ across conditions (Kruskal–Wallis $p = 0.024$), driven by the NEAT vs. ablation contrast. (b) Connection counts do not ($p = 0.244$).



(a) Best-genome node and connection counts over generations.



(b) Species count over generations.

Figure 8: Generational dynamics in the main study. Lines are across-seed means. Bands are the inter-quartile range. (a) Node counts diverge only after ≈ generation 300 and connection counts prune in lockstep. (b) Species counts saturate near the configured ceiling in all conditions.

5.4 Per-task trade-off and multi-task pressure

The aggregate fitness leaves open whether both tasks improved together or one task set the score throughout. To resolve this we plotted each champion's normalised Hopper score against its normalised Walker2D score and compared the per-task balance across conditions. Per-task scores are the normalised 20-episode returns from re-evaluation. The min of the two is the training objective. Episodes that share a genome are not independent observations, so all statistics in this section use seed-level means (one value per run, $n = 30$ per condition). Pooling episodes would overstate the effective sample size.

Champions in all three conditions clustered in a narrow vertical band at normalised Hopper ≈ 0.33–0.35, while Walker2D spread across a wider range and set the minimum for the large majority of genomes (Figure 9). Walker2D was the binding task for 97% of NEAT champions, 83% of HA-NEAT, and 90% of the ablation. The per-genome imbalance $|r_{\text{Hopper}}/3000 - r_{\text{Walker2D}}/5000|$ did not differ across conditions (medians 0.159 / 0.117 / 0.142; Kruskal–Wallis $H = 3.39$, $p = 0.184$, $n = 30$ per condition). The Hopper band sits at $0.34 \times 3000 \approx 1000$ raw reward, which for this environment corresponds to roughly 1000 timesteps of the per-step healthy/alive bonus (Freeman et al., 2021). A return in this band is consistent with a policy that survives the episode while making little forward progress, so the champions may effectively hop in place on Hopper. We did not render episodes to verify this. The evolved controllers keep both bodies upright while moving them only a little. Their returns stay

well below those that standard reinforcement learning methods such as SAC and TD3 reach on these tasks (Fujimoto et al., 2018; Haarnoja et al., 2018). Across every condition, then, champions solved the multi-task problem with the same unbalanced profile: a saturated Hopper score and a Walker2D score that carried the aggregate.

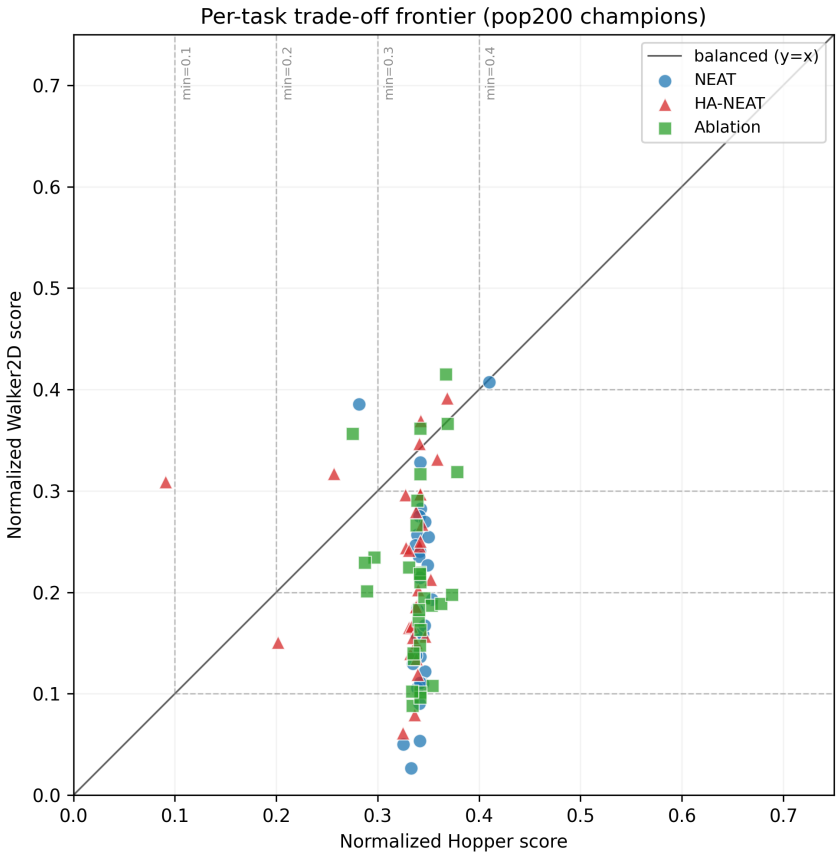


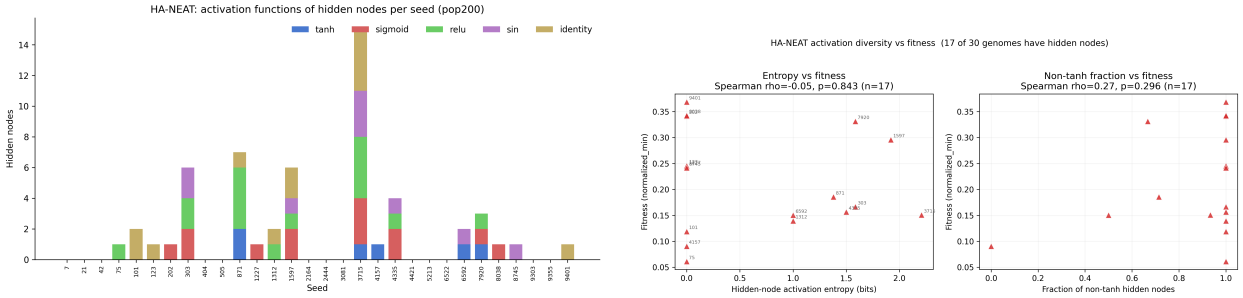
Figure 9: Per-task trade-off frontier in the main study ($n = 30$ per condition). Each point is a champion's normalised Hopper score (x) against its normalised Walker2D score (y). Champions cluster in a vertical band at Hopper $\approx$ 0.33–0.35 in all conditions. Walker2D is the binding task for 83–97% of champions. Per-genome imbalance does not differ across conditions (Kruskal–Wallis $p = 0.184$).

5.5 Activation-function usage within HA-NEAT

If activation diversity is the ingredient that HA-NEAT contributes, its champions should both use the non-tanh activations and be rewarded for doing so. We examined the activation composition of HA-NEAT's hidden nodes and its relationship to fitness. Of the 30 HA-NEAT champions, 17 evolved at least one hidden node. The remaining 13 had none, leaving 55

hidden nodes across the informative seeds.

The evolved activations were diverse and rarely defaulted to tanh (Figure 10a). Among the 55 hidden nodes, ReLU accounted for 27%, sigmoid for 24%, identity for 22%, sin for 16%, and tanh for 11%. Neither summary of diversity predicted fitness across the 17 informative champions (Figure 10b): activation entropy showed no monotonic relationship with fitness (Spearman $\rho = -0.05$, $p = 0.84$), nor did the fraction of non-tanh hidden nodes ($\rho = 0.27$, $p = 0.30$). With only 17 informative genomes these correlation tests are underpowered. We therefore read the flat correlations as an absence of evidence, not as evidence that diversity is irrelevant.



(a) Activation composition of the 55 HA-NEAT hidden nodes.

(b) Activation entropy vs. fitness ($n = 17$).

Figure 10: Activation usage within HA-NEAT champions in the main study. (a) Diversity is genuinely exploited (tanh is the least common activation at 11%), but (b) neither entropy nor non-tanh fraction predicts fitness across the 17 champions with hidden nodes.

5.6 Discussion

The three conditions reached statistically indistinguishable multi-task fitness (medians 0.184 / 0.208 / 0.199; $H = 1.48$, $p = 0.476$), yet they did not arrive there with the same genome structure. Activation diversity therefore did not raise multi-task fitness. The HA-NEAT algorithm nonetheless left a structural signal the fitness comparison alone would have missed. The three analyses in this section were exploratory, run only after the pre-specified fitness comparison returned a null result, and we read them as hypothesis-generating. They also involve multiple tests across related outcome measures without a battery-wide correction, so the single significant omnibus result (hidden-node counts, $p = 0.024$) should be read with an elevated false-positive risk in mind.

The clearest structural signal is that NEAT champions carried more hidden nodes than

the ablation's, even though the two reached the same fitness. A naive "NEAT with one activation" expectation predicts the opposite, which suggests the ablation does not behave like standard NEAT. The ablation runs the HA-NEAT mutation operator. Even with a tanh-only activation set, the activation-mutation event still fires and still reassigns the historical markers on every connection of the chosen hidden node. The activation change is a functional no-op (tanh to tanh), yet the marker side effect persists: reassigned markers register as disjoint genes in the compatibility distance and degrade crossover alignment for any genome carrying hidden nodes. We hypothesise that this suppresses the spread of hidden-node structures. That account would explain why the ablation tracks HA-NEAT, and not NEAT, in the node counts, and it would locate the compactness effect in the HA-NEAT algorithm instead of in activation diversity. We stress that this mechanism is a hypothesis. It is consistent with the code path and with the observed ordering, but we did not test it directly, and the effect is small and rests on a single significant pairwise contrast.

Within HA-NEAT, the evolved networks used the full activation set and did not collapse back to tanh, so the diversity mechanism is genuinely exercised. Even so, we detected no relationship between activation diversity and fitness, which is consistent with the headline null: on this task the mechanism HA-NEAT provides is exercised, and exercising it does not change fitness. The per-task analysis suggests why the task may be insensitive to that mechanism. In every condition the champions saturated Hopper at a low plateau and were bottlenecked by Walker2D. Once Hopper already exceeds Walker2D, the normalised-minimum objective makes further Hopper gains selectively invisible. We therefore read the effective optimisation problem as degenerating toward "improve Walker2D subject to keeping Hopper alive." That regime applies far less genuine multi-task pressure than the two-task framing implies, and it leaves little room for any mechanism to distinguish itself.

Taken together, these analyses answer the two research questions of Section 1: the shared genome learns a viable but weak joint controller, and activation diversity does not improve on it. Section 6 states both verdicts and their evidence in full.

These results speak to the gap identified in Section 2.9. HA-NEAT's advantages were established on static classification and regression benchmarks, where heterogeneous networks matched homogeneous baselines with 50–70% fewer nodes and connections (Hagg et al., 2017). Whether they transfer to a temporally extended control task was the open question this thesis tested. On the fitness criterion the answer here is no: in this multi-task locomotion setting, activation diversity produced no measurable advantage, even though the champions demonstrably used the full activation set (Section 5.5). The parsimony direction

did reappear, with both HA-NEAT variants ending at fewer hidden nodes than NEAT (significantly so only for the ablation), but the ablation complicates its source: the compactness persisted without any actual activation diversity, which points to the mutation mechanism as the driver, a separation the original single-algorithm comparisons could not make. The question of whether per-neuron functional roles modularise the shared policy (Section 2.9) therefore finds no support at the fitness level. This verdict is only indirect, since we measured modularity through fitness and activation-usage statistics alone. We therefore cannot conclude that no modularity exists, and the question stays open (Section 6). The collapse toward single-task Walker2D optimisation weakens this verdict further: because the objective drifted toward one task, the design applied less multi-task pressure than intended, and a setting that keeps both tasks under selection remains the sharper test of the question.

6 Conclusion and Future Work

This thesis asked two questions. Can a single neuroevolved genome control two locomotion bodies of different shape, and do heterogeneous per-node activation functions help it do so? We built one network over the union of the Hopper and Walker2D interfaces (Freeman et al., 2021), 17 inputs and 6 outputs. We fitted each environment to that network by zero-padding Hopper's observation and slicing its actions from the first three outputs, so that no part of the architecture was reserved for either task (Section 3.2). Selection used the normalised minimum of the two per-task returns. We chose that objective because it cannot reward a specialist, and we calibrated it against the sum and the product before the main experiments began (Sections 3.4 and 4.4). On that interface we compared three conditions: standard NEAT, HA-NEAT, and a tanh-only HA-NEAT ablation that keeps the full mutation mechanism but has no activation diversity. Each ran at population 200 over 750 generations across 30 seeds, and we re-evaluated every champion over 20 held-out episodes per task (Section 5).

Can a single genome control both tasks? Only partially, and at a weak level of competence. Every condition produced controllers that scored on both bodies rather than collapsing onto one, so the shared interface is workable. A single genome does run on two environments of different shape, with no per-task heads, no fine-tuning, and no task conditioning. The level of control, however, is low. The median normalised-minimum fitness was about 0.20, and the profile behind that number is unbalanced. Champions cluster in

a narrow band at a normalised Hopper score of roughly 0.34, which corresponds to about 1000 raw reward, the level a policy earning little beyond the per-step alive bonus would reach. Walker2D binds the minimum for 83–97% of champions and carries the aggregate, reaching only about a fifth of its normalisation constant (Section 5.4). Our answer to RQ1 is a qualified no. The shared genome learns a viable but weak joint controller, and the shared interface by itself did not yield strong control of both tasks.

Does HA-NEAT improve multi-task performance over NEAT? No. The three conditions reached statistically indistinguishable fitness (medians 0.184 / 0.208 / 0.199; Kruskal–Wallis $H = 1.48$, $p = 0.476$), their learning curves stayed interleaved across all 750 generations, and every pairwise confidence interval straddled zero (Section 5.2). The null is not a failure of the mechanism to engage: HA-NEAT's champions used the full activation set, with tanh their least common hidden activation at 11% (Section 5.5). The networks do use the mechanism, but using it makes no difference to fitness on this task. The follow-up question is the more informative one. Would any advantage be attributable to activation diversity, or to the speciation mechanism that arrives with it? The only structural signal we found, that champions end with fewer hidden nodes than NEAT's, is significant for the *ablation* (Section 5.3), the condition that has no activation diversity at all. The effect therefore appears where diversity is absent, and we attribute it to the marker-reassignment mechanism, not to heterogeneity. That attribution remains a hypothesis. It is consistent with the code path and with the observed ordering, but we did not test the mechanism directly, and it rests on a single significant pairwise contrast.

We claim three contributions, each stated at the level the evidence supports. The first is the shared-genome interface: a zero-padded, output-sliced encoding that lets one NEAT genome run on two environments of different observation and action dimensionality, paired with a calibrated aggregation that cannot reward specialisation. The second is the controlled three-condition comparison this interface enables. We ran it at 30 seeds per condition on a GPU-native JAX/Brax/TensorNEAT stack and report a null with its detectable-effect bound made explicit: differences of Cohen's $d \geq 0.74$ are excluded, smaller ones are not. The third is the ablation that separates activation diversity from the mutation mechanism that carries it. The original single-algorithm evaluations of HA-NEAT could not make that separation, and without it the structural signal reported here could not be attributed to either component.

The analyses of Section 5.6 have clear limitations. First, the hidden-node comparison rests

on very small, heavily tied counts (medians of 0–2 hidden nodes), which weakens the rank test and inflates the influence of a few large genomes. We therefore read the significant NEAT-versus-ablation contrast as suggestive. Second, the species-count comparison is uninformative because all conditions saturate the configured species ceiling, so this design cannot test whether HA-NEAT's speciation protection sustains more diversity. Third, the Hopper plateau confounds the multi-task claim: because Walker2D is almost always the binding task, the study measures single-task Walker2D optimisation more than true multi-task pressure. Fourth, the activation-fitness correlations rest on only 17 informative genomes and are underpowered. Fifth, all of these results come from a single population size (200), so the structural effects may not persist at other scales. Sixth, we measured modularity through fitness and activation-usage statistics alone, with no per-task disabling of network parts or pathway analysis. Whether per-neuron activations modularise the shared policy therefore remains an open question.

Three concrete follow-ups target the first, third, and fifth of these limitations. A targeted ablation can test the marker-reassignment hypothesis: set the activation-mutation rate to zero, keep the rest of the HA-NEAT algorithm, and check whether the compactness effect disappears. Restoring genuine multi-task pressure calls for either a harder first task or an aggregation that does not clip improvements on the leading task. The normalised product is one such aggregation, since it keeps a nonzero selection gradient on both tasks throughout. It permits more between-task imbalance than the minimum does, as our calibration experiment showed (Section 4.4). Finally, repeating the complexity and activation analyses at larger population sizes would test whether the structural differences observed here are scale-dependent. None of these fitted the remaining compute budget, and all three are left to future work.

References

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/google/jax

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016, June 5). *OpenAI Gym*. arXiv: 1606.01540 [cs.LG]. https://doi.org/10.48550/arXiv.1606.01540

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, *6*(2), 182–197. https://doi.org/10.1109/4235.996017

Draganski, B., Gaser, C., Busch, V., Schuierer, G., Bogdahn, U., & May, A. (2004). Changes in grey matter induced by training. *Nature*, *427*, 311–312. https://doi.org/10.1038/427311a

Drion, G., O'Leary, T., & Marder, E. (2015). Ion channel degeneracy enables robust and tunable neuronal firing rates. *Proceedings of the National Academy of Sciences of the United States of America*, *112*(38), E5361–E5370. https://doi.org/10.1073/pnas.1516400112

Duan, Y., Chen, X., Houthooft, R., Schulman, J., & Abbeel, P. (2016, May 27). *Benchmarking Deep Reinforcement Learning for Continuous Control*. arXiv: 1604.06778 [cs.LG]. https://doi.org/10.48550/arXiv.1604.06778

Dubey, S. R., Singh, S. K., & Chaudhuri, B. B. (2022, June 28). *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*. arXiv: 2109.14545 [cs]. https://doi.org/10.48550/arXiv.2109.14545

Freeman, C. D., Frey, E., Raichuk, A., Girgin, S., Mordatch, I., & Bachem, O. (2021, June 24). *Brax – A Differentiable Physics Engine for Large Scale Rigid Body Simulation*. arXiv: 2106.13281 [cs.RO]. https://doi.org/10.48550/arXiv.2106.13281

Fu, J., Kumar, A., Nachum, O., Tucker, G., & Levine, S. (2020). *D4RL: Datasets for Deep Data-Driven Reinforcement Learning*. arXiv: 2004.07219. https://doi.org/10.48550/arXiv.2004.07219

Fujimoto, S., van Hoof, H., & Meger, D. (2018). *Addressing Function Approximation Error in Actor-Critic Methods*. arXiv: 1802.09477. https://doi.org/10.48550/arXiv.1802.09477

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. arXiv: 1801.01290. https://doi.org/10.48550/arXiv.1801.01290

Hagg, A., Mensing, M., & Asteroth, A. (2017, March 21). *Evolving Parsimonious Networks by Mixing Activation Functions*. arXiv: 1703.07122 [cs]. https://doi.org/10.48550/arXiv.1703.07122

Holtmaat, A., & Svoboda, K. (2009). Experience-dependent structural synaptic plasticity in the mammalian brain. *Nature Reviews Neuroscience*, *10*(9), 647–658. https://doi.org/10.1038/nrn2699

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*, *25*. Retrieved February 14, 2026, from https://proceedings.neurips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html

Leng, Y., & Xiong, D. (2025, January). Towards Understanding Multi-Task Learning (Generalization) of LLMs via Detecting and Exploring Task-Specific Neurons. In O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. D. Eugenio, & S. Schockaert (Eds.), *Proceedings of the 31st International Conference on Computational Linguistics* (pp. 2969–2987). Association for Computational Linguistics. Retrieved June 29, 2026, from https://aclanthology.org/2025.coling-main.200/

Nair, V., & Hinton, G. E. (2010). Rectified Linear Units Improve Restricted Boltzmann Machines.

Padmanabhan, K., & Urban, N. N. (2010). Intrinsic biophysical diversity decorrelates neuronal firing while increasing information content. *Nature neuroscience*, *13*(10), 1276–1282. https://doi.org/10.1038/nn.2630

Papavasileiou, E., Cornelis, J. P., & Jansen, B. (2022). Towards optimizing neural networks' connectivity and architecture simultaneously with feature selection. *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 667–670. https://doi.org/10.1145/3520304.3528794

Papavasileiou, E., & Jansen, B. (2017). The importance of the activation function in NeuroEvolution with FS-NEAT and FD-NEAT. *2017 IEEE Symposium Series on Computational Intelligence (SSCI)*, 1–7. https://doi.org/10.1109/SSCI.2017.8285328

Pascual-Leone, A., Amedi, A., Fregni, F., & Merabet, L. B. (2005). THE PLASTIC HUMAN BRAIN CORTEX. *Annual Review of Neuroscience*, *28*, 377–401. https://doi.org/10.1146/annurev.neuro.27.070203.144216

Perez-Nieves, N., Leung, V. C. H., Dragotti, P. L., & Goodman, D. F. M. (2021). Neural heterogeneity promotes robust learning. *Nature Communications*, *12*(1), 5791. https://doi.org/10.1038/s41467-021-26022-3

Ramachandran, P., Zoph, B., & Le, Q. V. (2017, October 27). *Searching for Activation Functions*. arXiv: 1710.05941 [cs]. https://doi.org/10.48550/arXiv.1710.05941

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by backpropagating errors.

Stanley, K. O. (2007). Compositional pattern producing networks: A novel abstraction of development. *Genetic Programming and Evolvable Machines*, *8*(2), 131–162. https://doi.org/10.1007/s10710-007-9028-8

Stanley, K. O., Clune, J., Lehman, J., & Miikkulainen, R. (2019). Designing neural networks through neuroevolution. *Nature Machine Intelligence*, *1*(1), 24–35. https://doi.org/10.1038/s42256-018-0006-z

Stanley, K. O., D'Ambrosio, D. B., & Gauci, J. (2009). A Hypercube-Based Encoding for Evolving Large-Scale Neural Networks. *Artificial Life*, *15*(2), 185–212. https://doi.org/10.1162/artl.2009.15.2.15202

Stanley, K. O., & Miikkulainen, R. (2002). Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation*, *10*(2), 99–127. https://doi.org/10.1162/106365602320169811

Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 5026–5033. https://doi.org/10.1109/IROS.2012.6386109

Tripathy, S. J., Padmanabhan, K., Gerkin, R. C., & Urban, N. N. (2013). Intermediate intrinsic diversity enhances neural population coding. *Proceedings of the National Academy of Sciences of the United States of America*, *110*(20), 8248–8253. https://doi.org/10.1073/pnas.1221214110

Tutuncuoglu, B. T. (2025, February 11). *Neuron-Level Activation Learning in Neural Networks: A Self-Configuring AI Approach*. 5215137. https://doi.org/10.2139/ssrn.5215137

Wang, L., Zhao, M., Liu, E., Sun, K., & Cheng, R. (2024). Tensorized NeuroEvolution of Augmenting Topologies for GPU Acceleration. *Proceedings of the Genetic and Evolutionary Computation Conference*, 1156–1164. https://doi.org/10.1145/3638529.3654210

Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin*, *41*(4), 39–45. https://mlflow.org

Zhang, B., Zhang, W., Feng, Z., Xiao, W., Sun, J., Chen, J., & Wang, G. (2026, February 1). *Mixture-of-World Models: Scaling Multi-Task Reinforcement Learning with Modular Latent Dynamics*. arXiv: 2602.01270 [cs]. https://doi.org/10.48550/arXiv.2602.01270

Zhang, Y., & Yang, Q. (2021, March 29). *A Survey on Multi-Task Learning*. arXiv: 1707.08114 [cs.LG]. https://doi.org/10.48550/arXiv.1707.08114